

Scalable Multi-Agent Surveillance: A Kernel-Based Approach

Shashwata Mandal¹ and Sourabh Bhattacharya¹

Abstract—In this work, we address the deployment problem for a team of mobile guards that tries to maintain a line-of-sight with an unpredictable mobile intruder. First, we present a computationally efficient strategy for generating a set of points, called *kernel points*, that covers the entire polygon. We then introduce a polygon partitioning technique based on the location of the kernel points. Next, we propose control laws for a free guard to track an intruder in general polygonal environments based on the analysis of a pursuit-evasion game around a single corner [1]. Finally, we present several variations of the proposed control laws that include capture and search, and illustrate the improvement in the overall visual footprint of the team of mobile guards based on extensive simulations.

I. INTRODUCTION

Recent advances in robotics and AI are revolutionising business, society and our personal lives. Robots have been extensively deployed by law enforcement agencies and first responders to surveil areas alone or with an officer [2]. Based on data collected by the Electronic Frontier Foundation [3], more than 1,400 police departments use drones in some form. An important task undertaken by robots in such scenarios is that of reconnaissance. *Target tracking* is an important facet of reconnaissance. Target tracking refers to the task of planning the motion of an autonomous mobile guard in the presence of obstacles to visually track an unpredictable target. In this work, we address the problem of planning the motion of a team of autonomous mobile guards trying to track a single intruder.

In visibility-based coverage problems involving multiple guards, the primary approach has been to partition the environment into convex regions, and subsequently deploy a guard in each partition. Prior research in polygon partitioning has **primarily focused** on triangles, rectangles, and convex polygons [4][5] as basic shapes for partitions. Efforts on non-convex divisions and multiply-connected polygons are rather limited due to the inherent computational hardness. The task of segmenting a multiply connected polygon into star-shaped polygons, which may overlap, has been shown to be NP-hard [6]. In [7], authors use dynamic programming to segment a simply connected polygon into the smallest possible number of star components, notably omitting the use of Steiner points. Central to the aforementioned works is the concept of a *geometric kernel*. Roughly speaking, the kernel of a closed geometric shape is the locus of the points internal to it from which the whole shape is visible. The concept of *geometric kernel* of a polygon [8], a polyhedron, or more generally of a shape, is a pillar of computational geometry.

Apart from application in visibility-based computations for robot motion planning, the concept of kernels helps to plan the tool-path in NC machining [9], additive manufacturing [10] and finite element analysis [11].

The classical definition of kernel significantly restricts the class of polygons¹ that admit a non-empty kernel [12]. In [13], we introduce the concept of *distributed kernel* that generalizes the concept of kernel to all polygons. However, generalization comes at the cost of computation. Whereas, kernels can be computed in $O(n)$ [14] for star-shaped polygons, the problem of computing a distributed kernel of minimum size is computationally hard [15]. In [15], we presented an “implementable” technique based on associative rule mining [16][17] to compute a distributed kernel of minimum size. However, the worst-case complexity of the technique proposed in [15] is still exponential as the original problem is NP-hard. In our current work, we present a computationally efficient approach to generate a distributed kernel. In contrast to [15][13] which propose deployment of guards constrained to move on a segment between fragments of the distributed kernel, this work explores the problem of deploying a team of free guards (not constrained to move on a pre-defined path). The distributed kernel plays an important part in partitioning the polygon for guard allocation.

The contributions of this work are as follows: (i) We present a computationally efficient strategy for generating the distributed kernel of a polygon. The fragments of the kernel jointly ensure complete coverage of the polygon. (ii) We present a polygon partitioning technique based on the location of the fragments of the kernel. (iii) We propose control laws for a free guard to track an intruder in general polygonal environments. We present several variations of the proposed control laws that include capture and search to improve the overall visual footprint of the team of mobile guards.

This paper is organized as follows: Section II introduces the formulation of the problem, offering a foundation for the subsequent discussion. Section III provides a broadened perspective on the kernel concept when applied to arbitrary polygons. Section IV discusses a threat-based analysis, which is then utilized to derive a tracking controller for the guards. Section V demonstrates the effectiveness and performance of our proposed methods through comprehensive simulations. Finally, Section VI summarizes our conclusions and suggests directions for future research.

¹Department of Computer Science, Iowa State University, Ames, IA 50010, USA smandal@iastate.edu, sbhattac@iastate.edu

¹Polygons that admit a non-empty kernel are called star-shaped [12]

II. PROBLEM FORMULATION

We consider a scenario in which a team of mobile agents, referred to as *guards*, is deployed in an environment. The environment is assumed to be a simply-connected polygon with reflex vertices². In addition to the guards, we assume that a mobile agent, referred to as *intruder*, is present inside the polygon. We assume that each guard has an omni-directional field-of-view. Due to the presence of reflex vertices, the intruder can hide behind them, thereby breaking the line-of-sight with the guards. We say that the team of guards *tracks* the intruder if there is an unobstructed line-of-sight between the intruder and at least one guard. We assume that the intruder is visible to at least one guard in the beginning (i.e., at $t = 0$). Moreover, the intruder is assumed to be unpredictable, i.e., the guards do not possess information about the future actions of the intruder³. Given the aforementioned scenario, this work proposes deployment and motion strategies for the team of guards to track the intruder inside the polygonal environment. The foundations of our techniques are primarily based on novel computational geometric ideas for static coverage initially introduced in [15], [13], and optimal control for provably persistent tracking strategies [1].

III. STATIC COVERAGE

In this section, we present a technique for determining a set of strategic points that ensure complete visual coverage of a polygonal area. These points serve as anchors for the tracking strategy proposed in the next section. First, we define important geometric features of a polygon related to visibility-based computations.

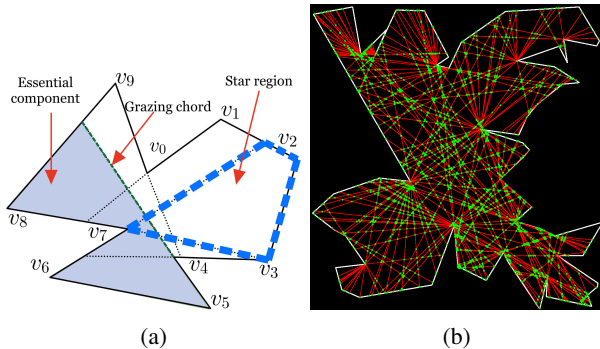


Fig. 1: (a) Figure shows a polygon with a reflex vertex(v_7) along with its star region (in pink) (b) Figure illustrating the candidate set points in a polygon in green

A polygon P is defined as a counterclockwise sequence of n vertices $v_0, \dots, v_{n-1}$, where each consecutive pair of vertices (v_i, v_{i+1}) is linked by an edge, and no interior intersections occur between any two edges. Two points are considered mutually visible if a line segment connecting

²A simply-connected polygon has no holes. Reflex vertices are vertices of the polygon that have internal angle greater than 180°

³Since the agents are assumed to be holonomic, the actions of the intruder are magnitude and direction of its velocity

them does not intersect the polygon's boundary. A vertex in the polygon is termed *reflex* if the internal angle at that vertex exceeds 180° . For the purposes of this discussion, we will use the terms *reflex vertex* and *corner* interchangeably. An edge is identified as a reflex edge if it is incident to a reflex vertex. For each reflex edge $e_i = (v_i, v_{i+1})$, a chord that exactly overlaps with e_i can be constructed, termed a *grazing chord*. The region inside the polygon, bounded by the grazing chords associated with a reflex vertex and visible from that vertex, is called the **star region** associated with the reflex vertex. Figure 1a displays the reflex vertices (v_0, v_4, v_7) of a polygonal environment and its grazing chords. We define the section of the polygon that contains the reflex edge e_i , divided by a grazing chord associated with e_i , as the essential component. For a point $p_0 \in P$, the *visibility polygon* of p_0 is defined as $VP(p_0) = \{p \in P, p \text{ is visible to } p_0\}$.

Algorithm 1 Concise Star Region Generator

Input: Q polygon, C corners, κ candidate set points

Output: SR set of star regions

```

1: function SIMPLIFYGEN( $Q, C, \kappa$ )
2:   Initialize  $SR$  for each  $j \in C$  using  $VP(j)$  and edge
   extensions
3:    $Q' \leftarrow Q, C' \leftarrow C$ 
4:   while  $|C'| > 1$  do
5:     Identify  $u$  and  $x$  as mutually visible reflex vertices,
     with  $w$  being the last non-reflex vertex preceding  $x$  in
     the vertex sequence, visible from  $u$ , ensuring no reflex
     vertices lie between  $u$  and  $x$ .
6:     Update  $SR(u)$  with  $VP(w)$  intersection
7:     Exclude vertices from  $u$  to  $w$  in  $Q'$ , keeping  $u$ .
8:     Remove  $u$  from  $C'$  if it's no longer a corner in  $Q'$ 
9:     if  $|C'| \leq 2$  then
10:      Update  $SR(u)$  with  $SR(u) \cap Q'$ 
11:    end if
12:  end while
13:  for each  $p \in \kappa$  do
14:    for each  $j \in C$  do
15:      Assign  $p$  to  $SR(j)$  if  $p \in SR(j)$ 
16:    end for
17:  end for
18:  return  $SR$ 
19: end function

```

The set of points, referred to here as the *candidate set*, is constructed as follows. For a corner v_i and a visible vertex v_j , extending the line $v_j v_i$ to intersect the polygon boundary at v_k forms a segment $v_i v_k$, entirely within the polygon. We designate F as the collection of these segments $v_i v_k$. The candidate set κ comprises of the intersection points of segments in F with each other and the polygon's boundary, as illustrated in Figure 1b. Points in κ encode critical changes in visibility with respect to the vertices of the polygon. Next, we present an algorithm that uses the candidate set for generating a set of points that ensure coverage of the entire polygon.

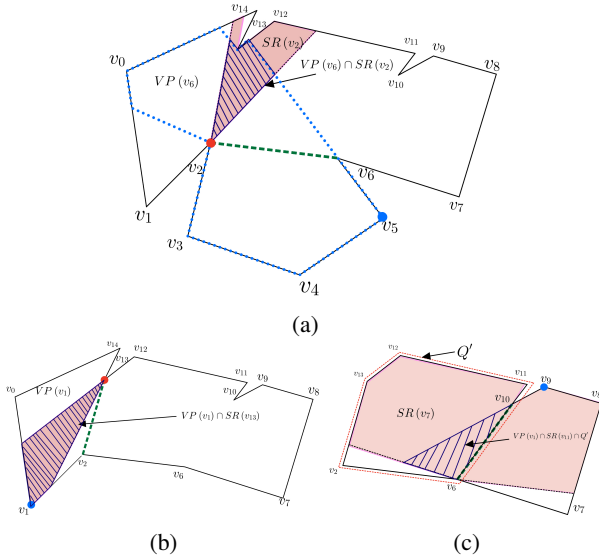


Fig. 2: Figures 2a, 2b and 2c shows the stepwise polygon reduction (inorder) of P . The blue dotted line symbolizes the visibility polygon of v_{12} while the hashed region shows the modified star region of c .

Our algorithm operates in two phases. Algorithm 1 identifies regions within a polygon that ensure complete visual coverage [13]. Then, Algorithm 2 refines these regions into a concise set of observation points from potential candidates, as shown in Figure 2. The process begins by establishing star regions for each corner (line 2), then selects pivot and supplementary reflex vertices u and x based on visibility criteria (line 5). For example, Figures 2a to 2c illustrate sequential pivot selections and corresponding vertex eliminations to reduce the polygon. This includes updating the star region by the visibility polygon of vertex w (line 6) and iteratively removing vertices to redefine the polygon's boundaries. The output of the algorithm is a reduced polygon marked by a red dashed line in Figure 2c, with each point in κ assigned to a star region if it lies within it (line 15), producing a set of star regions SR linked to κ .

Algorithm 2 generates a set of coverage points within the polygon, fewer in count than the polygon's corners, using the sets SR and κ from Algorithm 1. The set L maps candidate points to their associated corners. Initially, candidate points are linked to their corresponding star regions (line 6). In Figure 3a, two points (red and blue) in κ are shown. The red point lies at the intersection of $SR(v_2)$ and $SR(v_{13})$, and the blue point at the intersection of $SR(v_6)$ and $SR(v_{10})$. Thus, v_2 and v_{13} are included in L for the red point, and v_6 and v_{10} for the blue point. A point x with the maximum associated star regions is chosen and added to the cover (lines 13-14), then removed from L (line 16), along with all instances of its associated star regions (line 18). In Figure 3a, both the red and blue points are linked to two-star regions each. This process repeats until all star regions are covered. The computational complexity of Algorithm 2 is $O(N^3 n^2 \log(N^2 n^2))$, where N and n represent the number of corners and vertices of the polygon. Algorithm 1 ensures that

Algorithm 2 Candidate Set Reduction

Input: Q polygon, SR star regions, κ candidate set points

Output: J kernel set

```

1: function STARGEN( $Q, SR, \kappa$ )
2:    $L \leftarrow \{(x, \{\}) \mid x \in \kappa\}$ 
3:   for  $x \in \kappa$  do
4:     for  $c \in SR$  do
5:       if  $x \in SR(c)$  then
6:          $L_x.APPEND(c)$ 
7:       end if
8:     end for
9:   end for
10:   $J \leftarrow \emptyset$ 
11:   $C \leftarrow \emptyset$ 
12:  while  $|C| \neq N$  do
13:     $(x, L_x) \leftarrow$  entry in  $L$  with maximum cardinality
14:    add  $x$  to  $J$ 
15:     $C \leftarrow C \cup L_x$ 
16:    Remove  $(x, L_x)$  from  $L$ 
17:    for all  $(y, L_y) \in L, L_y \leftarrow L_y \setminus L_x$ 
18:  end while
19:  return  $J$ 
20: end function

```

the polygon can be covered if a point exists within each star region. Algorithm 2 selects a point with the most star regions, adds it to the cover, and removes those regions from further consideration, continuing until all regions are covered. This results in the kernel set, composed of **kernel points**.

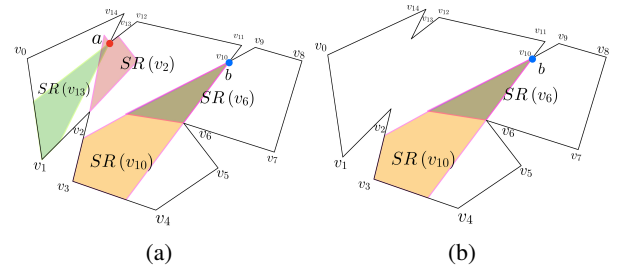


Fig. 3: Figure shows a set of overlapping star regions in which $a, b \in \kappa$ are points lying in two-star regions each.

Figure 4a shows the correlation between the number of points in κ (generated by Algorithm 2), and the number of reflex vertices in the polygon based on 1000 random polygons. From the plots, we can infer that the required number of static guards, as per Algorithm 2, is approximately $0.33N$, where N denotes the total number of reflex vertices of the polygon. Figure 4b compares our algorithm's runtime to the optimal Apriori solution in [13]. Hence the algorithm presented in this work provides an implementable solution, albeit at the cost of optimality.

IV. TRACKING WITH MOBILE GUARDS

In this section, we present a motion strategy for mobile guards to track a mobile intruder. This allows the tracking

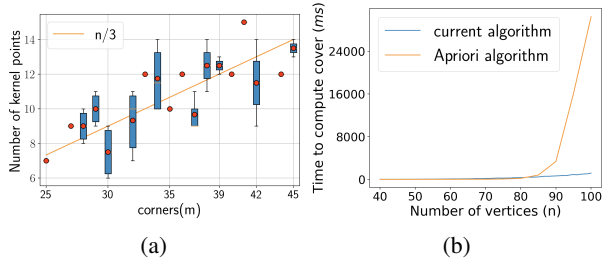


Fig. 4: Figure 4a shows a plot of the number of kernel points computed by Algorithm 2 against the number of corners in the polygon. The box and whisker subplots shows the range of values of kernel points for each corner size. Figure 4b shows a plot of the time taken by Algorithms 1 and 2 to compute the coverage points vs the time taken by the apriori-based algorithm proposed in [13] to do the same.

task to be accomplished when the number of guards is fewer than the number of kernel points (output of Algorithm 2). In [13], we presented a strategy for a line guard to track a mobile intruder. In our current work, we remove the constraint on the guards to move on straight lines, thereby, allowing them to move freely. First, we use the kernel points as a basis to partition the polygon for allocating guards. Next, we propose a tracking control law for the guard to track the intruder in its allocated partition.

A. Partitioning

Given a set of kernel points (output of Algorithm 2) and guards (less than the number of kernel points), we propose a clustering technique to partition the kernel points with an objective to minimize geodesic distances within each cluster⁴. By minimizing the geodesic distance within a cluster, we can restrict guards to specific areas of the polygon, effectively forming surveillance manifolds around each guard's location.

Agglomerative clustering [19] is a hierarchical technique that groups objects based on similarity. Figure 5 demonstrates this process on a polygon (from Algorithm 2). Starting with each observation as its own cluster, the method iteratively merges the closest clusters until a single cluster remains or a desired number is reached. A **dendrogram** (left side of Figure 5) visualizes this process as the threshold (y-axis) increases. The polygon at the bottom shows the initial 15 kernel points as 15 clusters. As the threshold rises, clusters merge. The two top polygons illustrate clustering for 7 and 9 clusters, with the horizontal line on the dendrogram marking the corresponding threshold value. Agglomerative clustering's adaptability to non-Euclidean distances makes it preferable to k-means for complex polygonal areas where the shortest path between two points may not be a straight line. Figure 6a shows kernel points clustered into 3 groups for 3 guards: (p_1, p_2) for guard 1, (p_3, p_4, p_5) for guard 2, and (p_6, p_7) for guard 3. Each guard is then responsible for

⁴Geodesic distances, determined through visibility graphs for the shortest path, are calculated in $O(V \log V + E)$ [18].

tracking the intruder within the union of visibility polygons of their assigned kernel points.

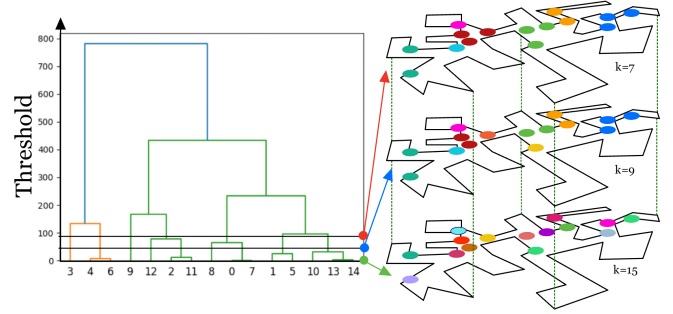


Fig. 5: Figure shows geodesic distance-based hierarchical clustering of kernel points inside a polygonal environment. The polygons on the right show the different stages of clustering of the points for different values of cluster numbers (k). Kernel points with the same color lie in the same cluster.

The jurisdiction of guard i is defined as $J_i = \bigcup_{p_j \in B_i} VP(p_j)$, where B_i is the set of kernel points in the cluster associated with guard i . Guards i and k are said to be neighbors if $J_i \cap J_k \neq \emptyset$. Figure 6b shows the boundaries of the jurisdictions of each guard in the polygon shown in 6a. The yellow and green guards are neighbors as their jurisdictions intersect. Conversely, the red and yellow guards are not neighbors, as their jurisdictions do not intersect.

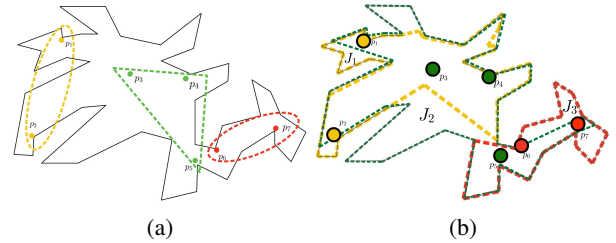


Fig. 6: Figure 6a illustrates the kernel points generated along with the clustered point for $k = 3$. Figure 6b illustrates the shared jurisdiction between the different clusters. J_1, J_2 and J_3 represent the jurisdiction for the yellow, green, and red guards respectively.

B. Active Tracking

In [1], authors analyze the tracking-problem as a pursuit-evasion game between an intruder (modeled as an evader) and a mobile guard (modeled as a pursuer) in an environment containing a single corner. Figure 7a shows the partition of the visibility polygon of an guard based on the outcome of the game and the strategy of the winner. The authors showed that the intruder can break the line-of-sight if it is within a distance $r_1 = r_p / \alpha$ (r_p is the distance of the guard from the corner and $\alpha = \bar{v}_e / \bar{v}_p$ is the ratio of the maximum speed of the intruder to that of the guard) from the reflex edge of the corner hidden from the guard. The guard has a strategy to track the evader in all other partitions. 7b shows the tracking strategy for the guard.

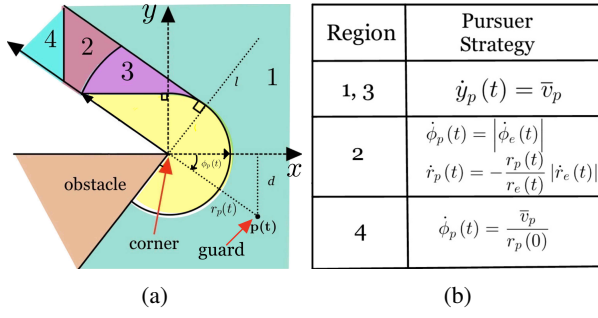


Fig. 7: Figure on the left shows the partition of the free space based on the strategy required by a mobile guard to track an intruder around a corner [1]. (x_p, y_p) and (x_e, y_e) denote the position of the pursuer and evader in cartesian coordinates, respectively. (r_p, ϕ_p) and (r_e, ϕ_e) denote the position of the pursuer and evader in polar coordinates, respectively.

In this work, we extend the aforementioned results to environments containing multiple corners. In the presence of multiple corners, we propose the following control law for the guard's motion:

$$\gamma(p, e) = \sum_{i \in C} \gamma_i(p, e) q_i(p, e), \quad (1)$$

where $\gamma_i(p, e)$ is a unit vector in the direction of the optimal control for the guard around corner i , $q_i(p, e)$ is a normalized weighting function that represents the **risk** of escape around corner i , and C is the set of corners in the visibility polygon of the guard.

Next, we propose a technique to compute $q_i(p, e)$. A gap-corner is a reflex vertex in the visibility polygon of the guard for which only one reflex edge is visible to the guard. Consider a list of gap corners sorted in the non-decreasing order of distance from the intruder. Let $l_0, \dots, l_{n'}$ denote the distance of the intruder from the gap-corners in the list where n' is the number of visible gap-corners. $q_i(p, e)$ is given by the following expression:

$$q_i(p, e) = \frac{f_q(l_i)}{\sum_j f_q(l_j)}, \text{ where} \quad (2)$$

where $f_q(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu_q}{\sigma_q}\right)^2}$, $\mu_q = l_0$, $\sigma_q = \frac{|l_0 - l_{n'}|}{2}$, $m \in \{1, \dots, n'\}$ and $x \in \{l_0, \dots, l_{n'}\}$.

V. SIMULATION RESULTS

In this section, we present simulation results for the control strategy proposed in the previous section. Figure 8 shows an intruder, modeled as a drone, navigating through the polygon within our simulation setup. In our simulations, the intruder implements the following adversarial strategy. Initially, it chooses a random corner in the environment. An RRT-based planner [20] is used to reach the chosen corner. Upon arriving at the corner, the intruder executes a sharp turn and proceeds to follow along a reflex edge connected to that corner for a brief period. This maneuvering pattern is systematically repeated to simulate the intruder's adversarial movement within the environment.

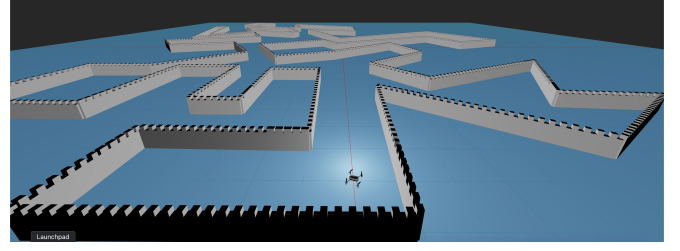


Fig. 8: The figure shows a snapshot of the simulation. The simulation is implemented in Python with C++ bindings for rapid rendering. The three-dimensional environment is created using ReactThreeFiber, enabling the simulation of guard visibility within the polygon.

A. Tracking

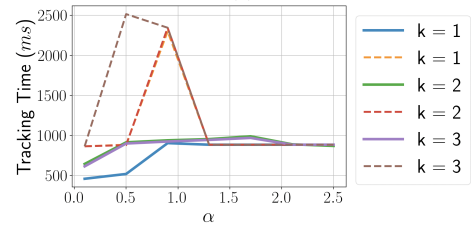
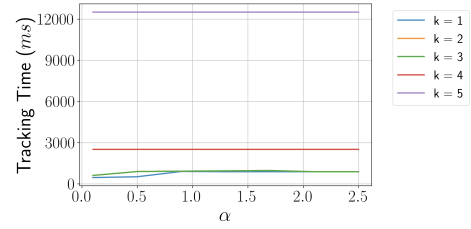


Fig. 9: Figure 9a shows a plot of the tracking time vs α for the control law proposed in (1). Figure 9b shows a plot of tracking time vs α for the control law proposed in (3)

The polygon in Figure 6a has 60 vertices, 29 corners, and a cover size of 15 as determined by Algorithm 2, where k represents the number of guards. Its narrow passageways and various compartments create substantial tracking challenges. Figure 9a compares the guard's performance with different numbers of guards. Initially, a line of sight between the guard and intruder is ensured. Performance for each technique (tracking and capture-assisted tracking) is measured by the maximum duration the intruder remains visible to at least one guard since the start of the simulation (*tracking time*). The plots show significant performance improvement as the number of guards increases.

Additionally, when the number of guards reaches 5, the duration until the initial loss of line-of-sight substantially increases, likely due to improved visibility handoff among adjacent guards.

B. Capture-assisted tracking

In this variant of the tracking strategy, we integrate capture as a supplementary mechanism within the tracking strategy

to improve the overall tracking time. We modify the control law proposed in (1) as follows:

$$\gamma(p, e) = h(p, e) \sum_{i \in C} \gamma_i(p, e) q_i(p, e) + (1 - h(p, e)) \bar{d}_e, \quad (3)$$

where $h(p, e)$ is a function that models the *threat* posed by the intruder to break the line-of-sight, and $\bar{d}_e$ is the unit vector from the guard to the intruder. The second term on the right hand side of (3) corresponds to the pursuit strategy for capture of the intruder. For our simulations, we model $h(p, e)$ as follows:

$$h(p, e) = \begin{cases} 0 & , x \geq \beta \\ 1 - e^{-\frac{1}{2} \left(\frac{x - \mu_h}{\sigma_h} \right)^2} & , x < \beta, \end{cases} \quad (4)$$

where $x = \frac{\bar{d}}{d_e}$, $\mu_h = \beta$, $\sigma_h = \frac{\beta}{0.8}$, $\bar{d}$ is the average of $\{1, \dots, n'\}$, d_e is the distance between the guard and intruder and β is the threshold ratio of distances between $\bar{d}$ and d_e . h represents the trade-off between tracking and capture depending on the position of the intruder and the position of the guard with respect to important features in the environment.

Figure 9b compares the tracking performance between two scenarios: tracking versus capture-assisted tracking. We maintain the same initial conditions in the simulations to compare the two techniques. For values of $\alpha < 1$, where α denotes the speed ratio of the guard to the intruder, integrating capture into the strategy grants the guard a notable advantage, reaching its peak when the speeds of both agents are similar. However, as the guard becomes slower than the intruder, a strategy focused solely on tracking yields better performance.

C. Capture-assisted tracking with search

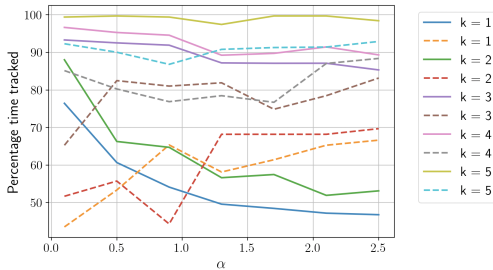


Fig. 10: Figure shows a plot of percentage time tracked(Φ) vs values of α . k represents the number of guards. Dotted lines represent capture-assisted tracking without search while solid lines represents capture-assisted tracking with search

In this section, we address the problem of intermittent tracking wherein the guards can lose line-of-sight with the intruder sporadically. We enforce a guard to implement a search strategy when both the following conditions are satisfied (i) The intruder is not in its line-of-sight. (ii) The intruder lies in its neighboring jurisdiction. The search strategy proposed for a guard utilizes the kernel points as patrol posts for visual coverage which in turn converts the tracking problem into a patrol and search problem [21][22]. The search

protocol directs it to proceed to the nearest kernel point within its cluster to patrol the graph to locate the intruder. We implement a patrolling strategy where each node tracks visit frequencies with a counter. The selection probability for the next visit is $1/f_i$, making less visited nodes more likely to be chosen. Guards decide their next node based on these probabilities, ensuring equitable coverage by prioritizing less frequented areas. Once the line-of-sight is re-established, the guard switches to a tracking mode.

We define the tracking efficiency (or the percentage time tracked) ϕ as

$$\Phi(\%) = \frac{\text{duration of intruder tracking}}{\text{time elapsed (in ms)}} * 100, \quad (5)$$

where **duration of intruder tracking** is the time for which an intruder remains in sight of at least one guard. Figure 10 illustrates the tracking performance across varying numbers of guards, comparing two strategies: capture-assisted tracking with search and capture-assisted tracking without search. To facilitate a meaningful comparison between these strategies, we modify the capture-assisted tracking in the following manner to obtain the capture-assisted tracking without search strategy. When the guards lose the line-of-sight with the intruder, they remain static until the line-of-sight to the intruder is re-established after which the tracking assisted capture strategy is reactivated. In capture-assisted tracking with search, the guards implement the search strategy once the visual contact with the intruder is lost (while implementing the tracking assisted capture strategy). The guard switches to the tracking assisted capture strategy once the visual contact is re-established during search. From Figure 10, we can conclude that augmenting the tracking strategy with search significantly enhances the tracking efficiency, likely because the guards make deliberate efforts to locate the intruder whenever the line of sight is broken, a key advantage of the search-based technique. This plot was generated using the average performance across 300 simulations while varying the number of guards and their paths.

VI. CONCLUSION AND FUTURE WORKS

This study presents a computationally efficient framework for visibility-based target tracking in polygonal environments, leveraging a kernel-based strategy for scalable multi-agent coordination. Future work will address motion constraints, sensing limitations, and environmental uncertainties to enhance real-world applicability. Extending the framework to 3D environments [23][24] will introduce challenges such as occlusions and terrain variations, requiring improved perception and tracking methods. Additionally, integrating SLAM, real-time multi-agent planning, and probabilistic tracking [25][26] will enhance robustness in dynamic settings. Exploring multi-agent collaboration strategies will further improve coordination, enabling efficient deployment in large-scale surveillance and security applications.

REFERENCES

- [1] S. Bhattacharya and S. Hutchinson, "A cell decomposition approach to visibility-based pursuit evasion among obstacles," *The International Journal of Robotics Research*, vol. 30, no. 14, pp. 1709–1727, 2011. [Online]. Available: <https://doi.org/10.1177/0278364911415885>
- [2] R. McCullom, "Robot police dogs are on patrol, but who's holding the leash?" *Undark*, 09 2023. [Online]. Available: <https://undark.org/2023/09/04/robot-police-dogs/>
- [3] N. Yancey-Bragg, "More than 1,400 police departments use drones in some form," *USA Today*, 10 2023. [Online]. Available: <https://www.usatoday.com/story/news/nation/2023/10/31/nyc-police-robots-privacy-safety-concerns/71218199007/>
- [4] D. Banerjee and R. Inkulu, "Vertex guarding for dynamic orthogonal art galleries," *arXiv.org*, 2021.
- [5] J. O'Rourke, *Art gallery theorems and algorithms / Joseph O'Rourke*, ser. International series of monographs on computer science ; 3, 1987.
- [6] J. O'Rourke and K. Supowit, "Some NP-hard polygon decomposition problems," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 181–190, 1983.
- [7] J. M. Keil, "Decomposing a polygon into simpler components," *SIAM J. Comput.*, vol. 14, pp. 799–817, 1985. [Online]. Available: <https://api.semanticscholar.org/CorpusID:26105634>
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, Third Edition*, 3rd ed. The MIT Press, 2009.
- [9] Z. Zhaoju, Z. Lin, J. Huang, L. Zheng, and B. He, "A digital twin-based machining motion simulation and visualization monitoring system for milling robot," *The International Journal of Advanced Manufacturing Technology*, vol. 127, pp. 1–13, 06 2023.
- [10] F. Calignano, D. Manfredi, E. P. Ambrosio, S. Biamino, M. Lombardi, E. Atzeni, A. Salmi, P. Minetola, L. Iuliano, and P. Fino, "Overview on additive manufacturing technologies," *Proceedings of the IEEE*, vol. 105, no. 4, pp. 593–612, 2017.
- [11] W. Xu and I. Neumann, "Finite element analysis based on a parametric model by approximating point clouds," *Remote Sensing*, vol. 12, no. 3, 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/12/3/518>
- [12] F. P. Preparata and M. I. Shamos, *Computational geometry: an introduction*. Berlin, Heidelberg: Springer-Verlag, 1985.
- [13] S. Mandal and S. Bhattacharya, "From distributed coverage to multi-agent target tracking," in *2023 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, 2023, pp. 36–42.
- [14] M. I. Shamos and D. Hoey, "Geometric intersection problems," in *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, 1976, pp. 208–215.
- [15] S. Mandal and S. Bhattacharya, "Relay pursuit for multirobot target tracking on tile graphs," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 7691–7698.
- [16] A. Evfimievski, R. Srikant, R. Agrawal, and J. Gehrke, "Privacy preserving mining of association rules," *Information systems (Oxford)*, vol. 29, no. 4, pp. 343–364, 2004.
- [17] G. Piatetsky-Shapiro, "Discovery, analysis, and presentation of strong rules," in *Knowledge Discovery in Databases*, 1991.
- [18] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Commun. ACM*, vol. 22, no. 10, p. 560–570, oct 1979. [Online]. Available: <https://doi.org/10.1145/359156.359164>
- [19] A. K. Jain and R. C. Dubes, *Algorithms for clustering data*. USA: Prentice-Hall, Inc., 1988.
- [20] S. M. LaValle *et al.*, "Rapidly-exploring random trees: A new tool for path planning," 1998.
- [21] X. Zheng, S. Jain, S. Koenig, and D. Kempe, "Multi-robot forest coverage," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2005, pp. 3852–3857.
- [22] K. Y. Lin, "Optimal patrol on a perimeter," *arXiv: Optimization and Control*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:148571836>
- [23] E. Lipka, "A note on minimal art galleries," 09 2019.
- [24] S. Lunz, Y. Li, A. W. Fitzgibbon, and N. Kushman, "Inverse graphics GAN: learning to generate 3d shapes from unstructured 2d data," *CoRR*, vol. abs/2002.12674, 2020. [Online]. Available: <https://arxiv.org/abs/2002.12674>
- [25] H. Surmann, A. Nüchter, and J. Hertzberg, "An autonomous mobile robot with a 3D laser range finder for 3d exploration and digitalization of indoor environments," *Robotics Auton. Syst.*, vol. 45, pp. 181–198, 2003. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15547002>
- [26] X. Xu, "Research on visual SLAM for dynamic scenes," *Applied and Computational Engineering*, vol. 12, pp. 34–38, 09 2023.